

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 Анализ требований	3
2 Разработка первой версии	4
2.1 Выбор архитектуры	4
2.2 Подготовка датасета	5
2.2.1 Выбор источника фотографий	5
2.2.2 Генерация искаженных изображений	7
2.2.3 Разделение датасета	11
2.3 Первая итерация	13
2.4 Вторая итерация	16
3 Разработка второй версии	20
3.1 Доработка архитектуры	20
3.2 Расширение датасета	21
3.2.1 Выбор источника фотографий	21
3.2.2 Генерация искаженных изображений	22
3.3 Третья итерация	22
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	28

ВВЕДЕНИЕ

В рамках преддипломной практики проводилась работа над рабочим проектом ВКРБ «Программная система для редактирования изображений с автоматическим отбором фотографий». Разрабатываемая система представляет собой приложение-редактор, дополненное модулем интеллектуальной оценки технического качества снимков. Назначение модуля - автоматизировать первичный отбор удачных кадров и сократить время пользователя на просмотр и сортировку фотоматериала.

Ежедневно пользователи делают большое количество фотографий, однако значимая часть кадров оказывается технически неудачной. При этом ручная оценка качества требует времени. В связи с этим актуальной является разработка решения, которое сможет определять распространённые технические дефекты.

Цель преддипломной практики - разработать нейронную сеть, которая автоматически выявляет технические дефекты фотографий.

Для достижения поставленной цели были сформулированы следующие задачи:

- 1) изучить предметную область автоматической оценки технического качества изображений и определить перечень целевых дефектов;
- 2) подготовить датасет для обучения модели;
- 3) спроектировать и реализовать архитектуру нейронной сети;
- 4) подобрать параметры обучения и метрики оценки;
- 5) провести валидацию качества и проанализировать результаты.

1 Анализ требований

Разрабатываемая в рамках ВКРБ система предназначена для редактирования изображений и автоматического отбора фотографий по техническому качеству. Следовательно, ключевое требование к нейросетевой части системы - способность устойчиво определять, является ли снимок технически удачным. Оценка должна быть применима к повседневным пользовательским фотографиям.

В контексте задачи отбора важно, чтобы результат работы нейронной сети был интерпретируемым и пригодным для использования в логике приложения: например, должна быть возможность настраивать пороги чувствительности. Также система должна быть рассчитана на практическое применение, поэтому требования включают стабильность работы на изображениях в распространённых форматах.

С учётом назначения приложения были выделены дефекты, которые наиболее часто встречаются в пользовательской съёмке, заметно снижают техническое качество и при этом могут быть формализованы для автоматического определения. Кроме того, выбранные дефекты должны быть релевантны для последующей интеграции в редактор и принятия решения об отбраковке кадра.

В результате научно-исследовательской работы, выполненной в 7 семестре, к системе в качестве целевых технических дефектов выбраны:

- 1) нерезкость (смаз/размытие) - в том числе из-за промаха фокуса и движения камеры/объекта;
- 2) недосвет - изображение слишком тёмное, с потерей деталей в тенях;
- 3) пересвет - слишком яркое изображение, с потерей деталей в светлых участках изображения.

Именно эти три класса дефектов используются в дальнейшем при подготовке данных, проектировании нейронной сети и оценке качества её работы.

2 Разработка первой версии

2.1 Выбор архитектуры

Для задачи автоматического отбора фотографий требовалось решение, которое одновременно устойчиво выделяет технические дефекты на разных сценах, достаточно лёгкое для интеграции в приложение и выдаёт отдельные оценки по каждому дефекту, чтобы затем можно было настраивать пороги и логику отбора изображений.

С учётом этого была выбрана архитектура по схеме «общая основа + несколько выходов». Общая часть (основа, бэкбон) извлекает универсальные признаки изображения, а далее идут отдельные выходные блоки (головы, логиты), каждый из которых отвечает за свой дефект. Такой подход особенно удобен в задаче, где дефекты могут встречаться независимо друг от друга: фотография может быть одновременно нерезкой и недоэкспонированной, либо иметь только один дефект. Поэтому вместо выбора одной оценки «хорошо»/«плохо» используется многометочная постановка: сеть выдаёт три независимые оценки - по нерезкости, недосвету и пересвету.

На практике входное изображение подаётся в основу EfficientNetB0 (предобученную на ImageNet, без верхней классификационной части), далее применяется глобальное агрегирование признаков и регуляризация (dropout). Затем от общего вектора признаков отходят три параллельных выхода, каждый из которых предсказывает один логит (одно число) - отдельно для нерезкости, недосвета и пересвета. Таким образом, начальная версия нейронной сети представляла собой EfficientNetB0 + 3 выхода по одному нейрону. На выходе сеть формирует три значения, каждое из которых отражает наличие или отсутствие соответствующего дефекта.

Архитектура схематично представлена на рисунке 1.

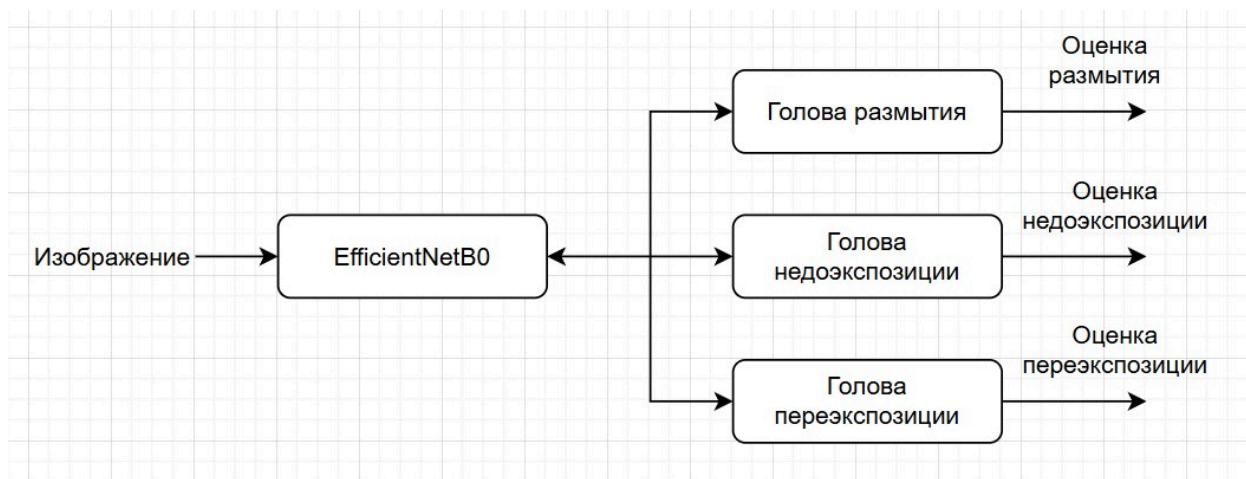


Рисунок 1 — Архитектура нейронной сети

Выбор EfficientNetB0 как основы был сделан на базе результатов научно-исследовательской работы, выполненной в 7 семестре. В рамках НИРа сравнивались несколько популярных архитектур по трём критериям: число параметров, вычислительная сложность и итоговое качество. Рассматривались, в частности, ResNet18, MobileNetV2, EfficientNet-B0 и более тяжёлая архитектура на трансформерах (TinyViT-21M). Сравнение показало, что тяжёлые модели действительно могут давать более высокую точность, но требуют существенно больше вычислений и ресурсов. При этом EfficientNet-B0 продемонстрировала лучший баланс: по качеству она заметно превосходила более простые варианты при умеренном размере и вычислительной нагрузке. Для задачи преддипломной практики этот компромисс оказался оптимальным: модель остаётся достаточно компактной для обучения, экспериментов и встраивания в десктопное приложение, но при этом обеспечивает хороший уровень точности и устойчивости.

2.2 Подготовка датасета

2.2.1 Выбор источника фотографий

Для обучения нейронной сети, определяющей технические дефекты фотографий, требуется набор данных с большим количеством изображений и корректной разметкой по целевым дефектам. При этом важно, чтобы разметка отражала именно техническое качество (нерезкость, недосвет, пересвет), а не смысл или «удачность» сцены. То есть разметка должна быть результатом не субъективной оценки «нравится»/«не нравится» опрошенных

людей. На практике получить такой набор данных непросто: ручная разметка тысяч фотографий требует значительных трудозатрат и остаётся частично субъективной, особенно в пограничных случаях.

При формировании данных возможны два подхода. Первый - использование реальных датасетов с оценками качества. Такие наборы ближе к реальным пользовательским условиям, однако часто содержат только общую оценку качества без явного указания типа дефекта. Кроме того, часть датасетов ориентирована на эстетическую (т.е. субъективную) оценку, что усложняет выделение конкретных технических причин брака.

Второй подход - формирование синтетического датасета на основе исходных качественных изображений, когда к «чистым» фотографиям программно добавляются контролируемые искажения (размытие, изменение яркости). В этом случае разметка формируется автоматически и не зависит от субъективной оценки, можно получить достаточное число примеров каждого дефекта, управлять силой дефекта через уровни искажений и обеспечивать воспроизводимость экспериментов.

С учётом ограничений преддипломной практики (необходимость быстро получить достаточный объём данных с однозначной разметкой) более рациональным является второй подход - обучение на синтетических искажениях.

В качестве источника данных выбран датасет KADIS-700K. Он содержит исходные изображения и большое число их версий с искусственно добавленными искажениями различных типов и уровней. Это позволяет сформировать выборку, напрямую соответствующую поставленной задаче: выделить только те типы искажений, которые моделируют нерезкость (размытие/смаз), недосвет и пересвет, и автоматически сформировать разметку для обучения нейронной сети.

KADIS-700K- крупный синтетический набор для оценки качества изображений. Он включает около 140000 исходных (неискажённых) изображений и примерно 700000 искажённых: для каждого исходного изображения подготовлено несколько вариантов, полученных добавлением

искажений, выбираемых из набора 25 типов с 5 уровнями выраженности. Набор искажений формируется с помощью кода генерации искажений в MATLAB, что позволяет точно знать тип и уровень добавленного дефекта.

В рамках ВКРБ рассматриваются только три дефекта, и поэтому в обучении не будут участвовать все 840000 изображений, но на их базе можно сформировать достаточную выборку изображений с нужными дефектами. Из всего спектра искажений KADIS-700K используются только следующие типы:

- 1) нерезкость/смаз: `lens blur (dist_type = 2)` и `motion blur (dist_type = 3)`;
- 2) пересвет: `brighten (dist_type = 16)`;
- 3) недосвет: `darken (dist_type = 17)`.

2.2.2 Генерация искаженных изображений

Для формирования обучающей выборки на основе KADIS-700K использовался исходный скрипт генерации, поставляемый вместе с кодом добавления искажений (`code_imdistort`) и функцией `imdist_generator`. В оригинальном варианте скрипт последовательно проходил по списку исходных изображений из файла `kadis700k_ref_imgs.csv`, считывал каждое изображение из каталога `ref_imgs/` и для заданного типа искажения генерировал все пять уровней (от 1 до 5). Исходный код добавления искажений представлен в листинге 1.

Листинг 1 — Исходный код добавления искажений

```
%% setup
clear; clc;
addpath(genpath('code_imdistort'));

%% read the info of pristine images

tb = readtable('kadis700k_ref_imgs.csv');
tb = table2cell(tb);

%% generate distorted images in dist_imgs folder

for i = 1:size(tb,1)
    ref_im = imread(['ref_imgs/' tb{i,1}]);
    dist_type = tb{i,2};
```

Продолжение листинга 1

```
        for dist_level = 1:5
            [dist_im] = imdist_generator(ref_im, dist_type,
dist_level);
            str = split(tb{i,1},'.');
            dist_im_name = [str{1} '_' num2str(tb{i,2},'%02d')
            '_' num2str(dist_level,'%02d') '.bmp'];
            disp(dist_im_name);
            imwrite(dist_im, ['dist_imgs/' dist_im_name]);
        end
    end
```

В рамках преддипломной практики данный скрипт был адаптирован под задачи ВКРБ. Во-первых, из полного набора искажений KADIS-700K были оставлены только те, которые соответствуют целевым дефектам. Для этого после чтения таблицы выполнялась фильтрация по `dist_type`, и дальнейшая генерация выполнялась только для типов 2, 3, 16 и 17.

Во-вторых, вместо перебора всех уровней искажения для каждого изображения уровни выбирались случайно из заранее заданных допустимых наборов, что позволило сконцентрировать данные на более заметных дефектах и избежать слабых искажений, которые хуже соответствуют практическим ситуациям. Использовались следующие диапазоны: для `dist_type` = 2 уровни [2, 3, 4], для `dist_type` = 3 уровни [4, 5], для `dist_type` = 16 уровни [4, 5], для `dist_type` = 17 уровни [4, 5].

В-третьих, изменён формат сохранения искажённых изображений. Вместо исходного расширения `.bmp` результаты сохранялись в `.jpg` с параметром качества 95. Такой выбор сделан для приближения данных к реальным пользовательским фотографиям (в большинстве случаев они хранятся в JPEG), а также для ускорения чтения с диска и уменьшения занимаемого объёма памяти. Имена файлов формировались по схеме «name_XX_YY.jpg», где к базовому имени исходного файла добавлялись код типа искажения XX и уровень YY, записанные в двухзначном формате.

Дополнительно в скрипт были внесены проверки, повышающие устойчивость генерации. Если целевой файл уже существует в `dist_imgs/`

(т.е. уже есть его искаженная версия), генерация пропускается, что позволяет безопасно перезапускать процесс без перезаписи уже сформированных данных. К тому же это позволило снизить избыточную корреляцию в данных: если сохранять множество вариантов одной и той же сцены, модель может переобучаться на характерные детали конкретного изображения и демонстрировать завышенные метрики вместо устойчивого распознавания дефектов на разных сценах.

Модифицированный код представлен в листинге 2.

Листинг 2 — Модифицированный код добавления искажений

```
%% setup
clear; clc;
addpath(genpath('code_imdistort'));

fid = fopen('labels.csv','w');
fprintf(fid,
'path,blur,under,over,dist_type,dist_level,ref\n');

tb = readtable('kadis700k_ref_imgs.csv');

% --- ОСТАВЛЯЕМ ТОЛЬКО НУЖНЫЕ ИСКАЖЕНИЯ ---
types = [2 3 16 17]; % gaussian/lens/motion blur +
brighten + darken
tb = tb(ismember(tb{:,2}, types), :); % 2-й столбец =
dist_type

tb = table2cell(tb);

ref_list = unique(tb(:,1));

for r = 1:numel(ref_list)
    ref_name = ref_list{r};
    ref_path = fullfile('ref_imgs', ref_name);

    % дефектов нет
    blur = 0; under = 0; over = 0;
    dist_type = 0; dist_level = 0;

    fprintf(fid, '%s,%d,%d,%d,%d,%d,%s\n', ref_path, blur,
under, over, dist_type, dist_level, ref_name);
end

%% generate distorted images in dist_imgs folder
```

Продолжение листинга 2

```
for i = 1:size(tb,1)
    ref_im = imread(fullfile('ref_imgs', tb{i,1}));
    dist_type = tb{i,2};

    if dist_type == 2
        allowed_levels = [2 3 4];
    elseif dist_type == 3
        allowed_levels = [4 5];
    elseif dist_type == 16
        allowed_levels = [4 5];
    elseif dist_type == 17
        allowed_levels = [4 5];
    else
        continue;
    end

    %% for k = 1:numel(keep_levels)
        dist_level =
allowed_levels(randi(numel(allowed_levels))));

        dist_im = imdist_generator(ref_im, dist_type,
dist_level);

        % имя файла остаётся совместимым: _тип_уровень.bmp
        strsr = split(tb{i,1},'.');

        dist_im_name = [strsr{1} '_' num2str(dist_type,'%02d')
'_' num2str(dist_level,'%02d') '.jpg'];
        out_path = fullfile('dist_imgs', dist_im_name);

        if exist(out_path, 'file')
            continue;
        end

        imwrite(dist_im, out_path, 'Quality', 95);

        % вычисляем метки по dist_type
        blur = 0; under = 0; over = 0;
        if dist_type == 2 || dist_type == 3
            blur = 1;
        elseif dist_type == 17
            under = 1;
        elseif dist_type == 16
            over = 1;
        end
    end
end
```

Продолжение листинга 2

```
        fprintf(fid, '%s,%d,%d,%d,%d,%d,%s\n', out_path, blur,
under, over, dist_type, dist_level, tb{i,1});

    %%    end
end

fclose(fid);
```

В результате сгенерированы искажённые изображения, соответствующие выбранным дефектам ВКРБ и сохраненные в формате, близком к условиям практического использования в приложении.

2.2.3 Разделение датасета

После генерации искажённых изображений был сформирован единый файл разметки labels.csv, содержащий пути к изображениям и целевые метки дефектов. Для дальнейшего обучения нейронной сети требовалось разделить данные на обучающую и валидационную выборки так, чтобы оценка качества была корректной и не завышалась из-за попадания в обе выборки разных версий одной и той же сцены.

Разделение «по строкам» в данном случае является нежелательным, так как в датасете присутствуют несколько вариантов одного исходного изображения (исходное и искажённая версии). Если часть вариантов одной сцены окажется в обучающей выборке, а другая часть - в валидационной, нейронная сеть может демонстрировать завышенные метрики за счёт узнавания особенностей конкретной сцены, а не за счёт устойчивого распознавания дефектов. Чтобы избежать такой утечки, разделение выполнялось не по отдельным файлам, а по имени исходного изображения.

В разметке для этого используется поле ref, которое хранит имя исходного файла и тем самым группирует все варианты одной сцены. В скрипте сначала загружается labels.csv, затем выполняется разделение по уникальным значениям ref. Множество исходных сцен случайно разбивается на две части в пропорции 80/20. После этого строки, относящиеся к выбранным

ref, собираются в train_df, а остальные - в val_df. Полученные таблицы сохраняются в файлы train.csv и val.csv.

Скрипт, разделяющий датасет на обучающую и валидационную выборки содержится в листинге 3.

Листинг 3 — Скрипт, разделяющий датасет на обучающую и валидационную выборки

```
import os
import pandas as pd
from sklearn.model_selection import train_test_split

LABELS_PATH = "labels.csv"
TRAIN_OUT = "train.csv"
VAL_OUT = "val.csv"
VAL_RATIO = 0.2
SEED = 42

df = pd.read_csv(LABELS_PATH)

df["path"] = df["path"].astype(str).str.replace("\\",
os.sep).str.replace("/", os.sep)

# Сплит по ref (чтобы не было утечки)
refs = df["ref"].astype(str).unique()
train_refs, val_refs = train_test_split(refs,
test_size=VAL_RATIO, random_state=SEED, shuffle=True)

train_df =
df[df["ref"].astype(str).isin(train_refs)].reset_index(drop=True)
val_df =
df[df["ref"].astype(str).isin(val_refs)].reset_index(drop=True)

train_df.to_csv(TRAIN_OUT, index=False)
val_df.to_csv(VAL_OUT, index=False)

print("Saved:", TRAIN_OUT, len(train_df))
print("Saved:", VAL_OUT, len(val_df))
print("Unique refs train/val:", len(train_refs),
len(val_refs))
```

В результате такой процедуры обеспечивается отсутствие пересечения сцен между выборками: одна и та же исходная фотография и её искажённая версия могут принадлежать только обучающей или только валидационной части.

2.3 Первая итерация

Обучение организовано через TensorFlow с использованием конвейера `tf.data`, так как разметка задаётся таблицами `train.csv` и `val.csv`, а не структурой папок вида класс/изображения. На этапе подготовки данных путь к файлу считывается из CSV, затем изображение загружается функцией `tf.io.read_file` и декодируется универсальным декодером `tf.image.decode_image`, который позволяет работать с форматами JPG/PNG/BMP. После декодирования изображение приводится к типу `float32` и нормируется в диапазон $[0; 1]$, а далее масштабируется до фиксированного размера 512x384 пикселей. Такой размер был выбран как компромисс между сохранением деталей дефектов (в первую очередь нерезкости) и ограничениями по ресурсам видеопамати при обучении. Для обучающей выборки дополнительно применяется перемешивание, затем данные группируются в пакеты (батчи) размером 16 изображений и подаются на обучение с предварительной подгрузкой, что уменьшает простои вычислений из-за ввода-вывода.

Выход основы (EfficientNetB0) представляет собой пространственную карту признаков, которая преобразуется в вектор с помощью глобального усредняющего пуллинга (`GlobalAveragePooling2D`). Далее применяется Dropout с вероятностью 0.2: этот слой на обучении случайно «отключает» часть нейронов и тем самым снижает риск переобучения. После этого реализованы три независимые головы - по одному нейрону на каждый дефект. Важно, что выходы голов являются логитами: логит - это значение до применения сигмоиды, а вероятность дефекта получается применением функции `sigmoid(logit)`.

В качестве функции потерь используется бинарная кросс-энтропия с параметром `from_logits=True`. Это означает, что функция потерь сама применяет сигмоиду внутри себя и корректно работает с логитами. Для контроля качества в процессе обучения применяется метрика AUC, причём в многометочном варианте. Бинарная кросс-энтропия рассчитывается по формуле (1).

$$L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)], \quad (1)$$

где $L(y, p)$ - значение функции потерь,

y - истинная метка ($y \in \{0, 1\}$),

p - предсказанная моделью вероятность класса 1 ($0 < p < 1$).

Первая итерация проводилась в два этапа. На первом этапе обучались только три выходных головы, а основа EfficientNetB0 была заморожена (`backbone.trainable=False`). Скорость обучения на этом этапе была выбрана 10^{-3} . На втором этапе основа размораживалась полностью (`backbone.trainable=True`) и выполнялось дообучение всей сети с уменьшенной скоростью обучения 10^{-4} . Уменьшение шага здесь принципиально: дообучение предобученной основы требует более аккуратных обновлений, иначе можно быстро получить деградацию признаков и нестабильность обучения. Размер пакета 16 выбран исходя из практического баланса: он достаточно велик для приемлемой стабильности градиентов, но при размере входа 512x384 не приводит к переполнению видеопамяти на видеокарте «домашнего» компьютера.

Динамика обучения показывает, что выбранная схема действительно даёт прирост качества. По итогам первого этапа (5-я эпоха обучения только голов) обучающая AUC достигла 0.6063 при значении функции потерь 0.4276, а на проверочной выборке AUC составила 0.7151 при `val_loss` = 0.4325. После перехода ко второму этапу и дообучения основы качество на проверке выросло существенно выше. В конце обучения достигнуты значения `auc` = 0.8048, `loss` = 0.3496, `val_auc` = 0.8627, `val_loss` = 0.3205. Таким образом, именно после дообучения основы модель значительно лучше отделяет дефектные изображения от качественных по всем трём меткам. Конкретные значения `loss` и `auc` в зависимости от эпох представлены на рисунке 2.

```

10000 00:00:17/1505824.639176 [320 device_compiler:n.196] Compiled cluster using XLA: This line is logged at
t once for the lifetime of the process.
2239/2239 ----- 2472s 1s/step - auc: 0.5786 - loss: 0.4334 - val_auc: 0.6434 - val_loss: 0.4439
Epoch 2/5
2239/2239 ----- 1938s 866ms/step - auc: 0.5939 - loss: 0.4298 - val_auc: 0.6979 - val_loss: 0.4363
Epoch 3/5
2239/2239 ----- 1822s 814ms/step - auc: 0.6014 - loss: 0.4286 - val_auc: 0.7220 - val_loss: 0.4347
Epoch 4/5
2239/2239 ----- 1739s 776ms/step - auc: 0.6056 - loss: 0.4279 - val_auc: 0.7121 - val_loss: 0.4358
Epoch 5/5
2239/2239 ----- 1675s 748ms/step - auc: 0.6063 - loss: 0.4276 - val_auc: 0.7151 - val_loss: 0.4325
Epoch 1/5
2239/2239 ----- 1755s 767ms/step - auc: 0.6616 - loss: 0.4142 - val_auc: 0.7355 - val_loss: 0.4448
Epoch 2/5
2239/2239 ----- 1719s 768ms/step - auc: 0.7457 - loss: 0.3812 - val_auc: 0.7357 - val_loss: 0.6894
Epoch 3/5
2239/2239 ----- 1741s 777ms/step - auc: 0.7776 - loss: 0.3647 - val_auc: 0.8399 - val_loss: 0.4188
Epoch 4/5
2239/2239 ----- 1853s 827ms/step - auc: 0.7931 - loss: 0.3562 - val_auc: 0.7491 - val_loss: 0.6603
Epoch 5/5
2239/2239 ----- 1765s 788ms/step - auc: 0.8048 - loss: 0.3496 - val_auc: 0.8627 - val_loss: 0.3205
Saved model to io_multihd_savedmodel

```

Рисунок 2 — Значения loss и AUC первой итерации

На графиках обучения заметна характерная проблема: проверочная функция потерь `val_loss` ведёт себя нестабильно и даёт резкие всплески на последних 5 эпохах, несмотря на общий рост AUC. Это поведение связано с тем, что при полном размораживании основы начинают активно обновляться слои пакетной нормализации `BatchNorm`. `BatchNorm` (пакетная нормализация) использует статистики текущего пакета и поддерживает скользящие оценки среднего и дисперсии, которые затем влияют на поведение модели на проверке и при реальном использовании. При небольшом размере пакета (в данном случае 16) оценки статистик получаются шумными, а их обновление может приводить к тому, что модель подстраивается под особенности обучающих пакетов и временно ухудшается на проверке. Визуально это проявляется скачками `val_loss` и колебаниями `val_auc`. В дальнейшем эта проблема была решена заморозкой `BatchNorm` на этапе дообучения основы. Графики обучения представлены на рисунке 3.

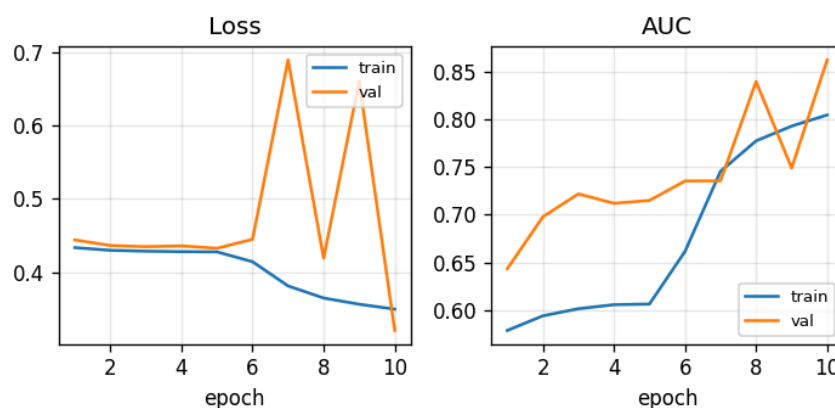
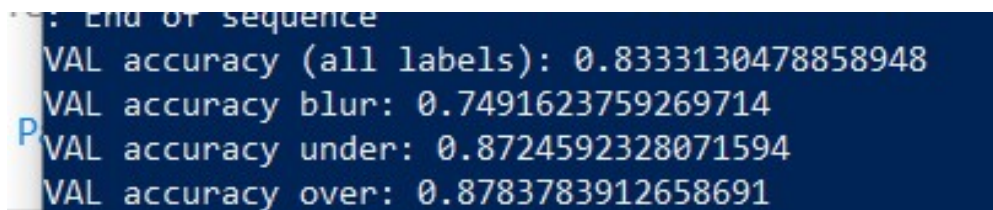


Рисунок 3 — Графики ключевых метрик первой итерации

Дополнительно качество было оценено отдельным скриптом на проверочной выборке в терминах точности (т.е. в более понятных человеку процентах угадывания) при фиксированном пороге решения. Получены значения: общая точность по всем меткам составила 0.8333, отдельно по дефектам - 0.7492 для blur, 0.8725 для under и 0.8784 для over. Результаты проверки точности представлены на рисунке 4.



```
...: End of sequence
VAL accuracy (all labels): 0.8333130478858948
VAL accuracy blur: 0.7491623759269714
VAL accuracy under: 0.8724592328071594
VAL accuracy over: 0.8783783912658691
```

Рисунок 4 — Точность правильных предсказаний первой итерации

В целом первая итерация подтвердила работоспособность выбранного направления: трёхголовая архитектура на EfficientNetB0 обучается на синтетических искажениях и демонстрирует высокую способность различать дефекты, а двухэтапная схема обучения существенно улучшает качество по сравнению с обучением только выходных слоёв. Одновременно были выявлены практические ограничения, связанные с нестабильностью проверки при дообучении основы из-за BatchNorm, что стало основанием для последующих улучшений во второй итерации модели.

2.4 Вторая итерация

Во второй итерации нейронной сети необходимо доработать первую итерацию так, чтобы убрать нестабильность на проверке, которую давали слои пакетной нормализации; усилить распознавание нерезкости как самой слабой части модели и сделать контроль обучения более прозрачным за счёт расширенного набора метрик и автоматического построения графиков.

В первой итерации каждая голова представляла собой 1 нейрон. По результатам первой проверки именно нерезкость распознавалась хуже остальных: точность по blur на проверке была заметно ниже (около 0.75), тогда как under/over были около 0.87–0.88. Это объяснимо: нерезкость проявляется в тонких деталях (высоких частотах), и для её отделения от требуется более сложная нелинейная комбинация признаков, чем для экспозиционных дефектов, которые сильнее зависят от распределения яркостей.

Поэтому во второй итерации для головы, отвечающей за размытие, добавлена цепочка слоёв $64 \rightarrow 32 \rightarrow 16 \rightarrow 1$ с ReLU-активацией и Dropout 0.15. Для under/over усиление более умеренное: $32 \rightarrow 16 \rightarrow 1$ и Dropout 0.1. Такой перекоп по мощности сделан осознанно: вычислительная цена растёт незначительно (эти слои стоят после глобального пуллинга и работают с вектором признаков), но модель получает возможность построить более сложную границу решения именно для blur - там, где первая итерация была слабее.

Вторая важная доработка касается устойчивости дообучения основы. В первой итерации при размораживании EfficientNet график функции потерь заметно «скакал», что связано с BatchNorm. Во второй итерации эта проблема решена двумя мерами одновременно. Во-первых, на этапе тонкой настройки BatchNorm-слои принудительно замораживаются (их параметры и статистики перестают обновляться). Во-вторых, размораживается не вся основа, а только её верхняя часть: все слои, кроме последних примерно 40, оставляются замороженными. Тем самым сохраняются универсальные низкоуровневые признаки (контуры, текстуры), а адаптация идёт за счёт верхних слоёв, которые отвечают за более сложные комбинации признаков. Дополнительно уменьшается скорость обучения на втором этапе до $5 \cdot 10^{-5}$, чтобы изменения весов были плавными и не приводили к деградации на проверке. В результате проверочные кривые становятся заметно более стабильными: на итоговом графике нет резких провалов val_loss, а к концу обучения значения сходятся к небольшим величинам. Графики обучения представлены на рисунке 5.

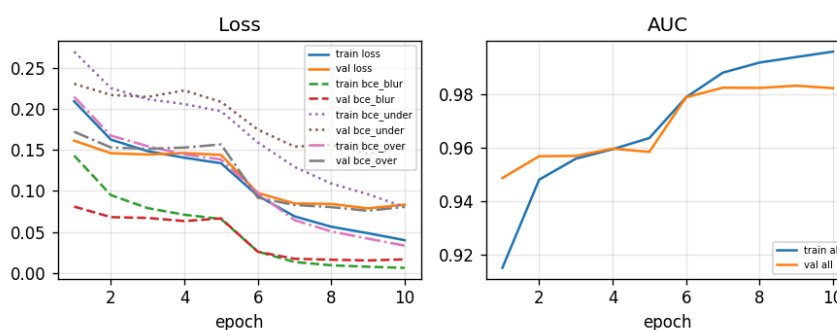
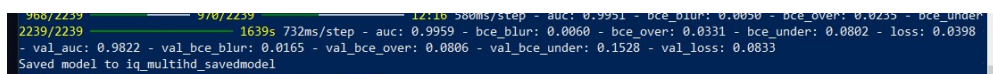


Рисунок 5 — Графики ключевых метрик второй итерации

Третья группа изменений - расширение набора метрик и улучшение журналирования. В первой итерации была только общая AUC, и по ней сложно понять, какая из голов учится хуже и почему. В конце концов скрипт по точности предсказаний дал понять, что самый слабый выход - выход по резкости, но такая проверка на выходе дает слабое представление о том, как каждая из голов обучалась. Во второй итерации добавлены метрики бинарной кросс-энтропии, посчитанной отдельно по каждому выходу. Новые графики также можно наблюдать на рисунке 5.

Финальные числа показывают, что доработки дали существенный прирост и по общим метрикам, и по каждой голове. В конце обучения получены значения: AUC на обучении 0.9959, AUC на проверке 0.9822, при этом проверочные потери составили $\text{val_loss} = 0.0833$. Финальные метрики представлены на рисунке 6.



```
988/2239 970/2239 12.16 samps/step - auc: 0.9951 - bce_blur: 0.0050 - bce_over: 0.0235 - bce_under: 0.0000
2239/2239 1639s 732ms/step - auc: 0.9959 - bce_blur: 0.0060 - bce_over: 0.0331 - bce_under: 0.0802 - loss: 0.0398
- val_auc: 0.9822 - val_bce_blur: 0.0165 - val_bce_over: 0.0806 - val_bce_under: 0.1528 - val_loss: 0.0833
Saved model to iq_multihd_savedmodel
```

Рисунок 6 — Значения loss и AUC второй итерации

Отдельный скрипт оценки на проверочной выборке показывает прикладные метрики качества при порогах 0.5 для всех голов. Общая точность по всем меткам (micro-accuracy) составила 0.9713, а точность строгого совпадения трёх меток одновременно (exact-match accuracy - доля изображений, где верно угаданы сразу все три метки) составила 0.9179. По дефектам получены следующие показатели. Для blur: precision 0.9946, recall 0.9866, F1 0.9906, accuracy 0.9953 (precision - доля правильных среди всех срабатываний, recall - доля найденных дефектов среди всех реальных дефектов). Для under: precision 0.7709, recall 0.7986, F1 0.7845, accuracy 0.9440 - видно, что именно under даёт основную часть ложных срабатываний (FP=271), поэтому его точность ниже. Для over: precision 0.9241, recall 0.8604, F1 0.8911, accuracy 0.9744, то есть пересвет распознаётся уверенно, но чуть хуже blur. Результаты проверки точности представлены на рисунке 7.

Приложение: precision - доля истинно-положительных среди всех предсказанных положительных, recall - доля найденных истинно-положительных

среди всех реальных положительных, F1 - гармоническое среднее precision и recall, а accuracy - доля верных предсказаний среди всех предсказаний.

```
[VAL METRICS]
val samples: 8954
thresholds: blur=0.5 under=0.5 over=0.5

Micro-accuracy (по всем меткам вместе): 0.9713
Exact-match accuracy (все 3 метки сразу): 0.9179

[BLUR]
TP=2216 FP=12 FN=30 TN=6696
pos=2246 neg=6708
precision=0.9946 recall=0.9866 f1=0.9906 accuracy=0.9953

[UNDER]
TP=912 FP=271 FN=230 TN=7541
pos=1142 neg=7812
precision=0.7709 recall=0.7986 f1=0.7845 accuracy=0.9440

[OVER]
TP=937 FP=77 FN=152 TN=7788
pos=1089 neg=7865
precision=0.9241 recall=0.8604 f1=0.8911 accuracy=0.9744
```

Рисунок 7 — Точность предсказаний второй итерации

Модель уже достигла высокого уровня разделения классов на проверке (val_auc 0.9822) и высокой практической точности по всем меткам (micro-accuracy 0.9713), а графики показывают выход метрик на плато без признаков систематического улучшения к концу 10-й эпохи. После стабилизации BatchNorm и частичного дообучения основы дальнейшие эпохи с большой вероятностью дадут убывающую отдачу и увеличат риск переобучения под синтетические искажения.

Отдельно стоит обсудить метрику under. У метки under много ложноположительных срабатываний (FP=271), особенно в сравнении с другими выходами. Поэтому её точность ниже (precision 0.7709 при accuracy 0.9440). Чаще всего это ночные/темные сцены: кадр намеренно тёмный, но модель воспринимает это как недосвет. Для over эта проблема тоже прослеживается, но не так явно, потому что «естественные» затемнения в реальном мире встречаются намного чаще сильных пересветов.

Таким образом, на второй итерации устранена нестабильность проверки, усилена самая слабая голова (blur), добавлен детальный контроль обучения по метрикам, а итоговые числа подсказывают, что выход under требует доработки.

3 Разработка второй версии

3.1 Доработка архитектуры

После экспериментов с первой версией модели была проведена доработка архитектуры. Базовая схема «одна общая часть + несколько выходов» была сохранена. Основное изменение связано с добавлением четвертой головы night, предназначенной для распознавания ночных сцен. Это связано с тем, что under может ошибочно срабатывать на ночных кадрах, где низкая яркость является нормой, а не дефектом. Голова night реализована как отдельный выход модели и используется в прикладной логике: при высокой вероятности ночной сцены фильтрацию по under можно ослаблять или отключать, чтобы не браковать корректные ночные фотографии - это должно помочь повысить точность головы under и снизить ее количество ложноположительных срабатываний непосредственно в прикладном приложении. Примечание - здесь и далее понятие «ночная сцена» не обязательно означает в прямом смысле ночную сцену (хотя и в том числе), но так же включает в себя темные и слабоосвещенные фотографии, сделанные, например, в условиях плохого освещения. Итоговая архитектура схематично представлена на рисунке 8.

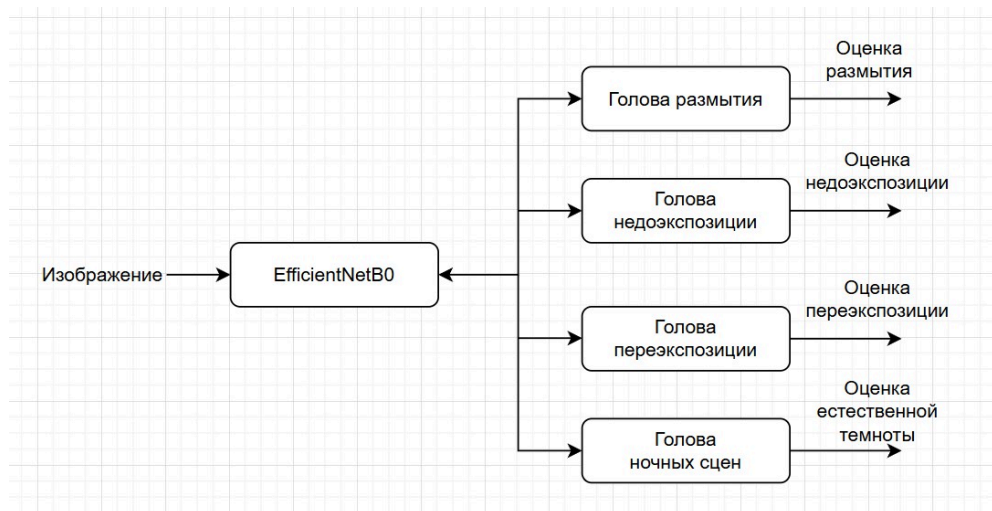


Рисунок 8 — Архитектура нейронной сети

От идеи дообучить under распознавать ночные сцены решено отказаться, потому что в прикладном приложении планируется разбивать фотографию большого разрешения на множество частей размером 512x384 пикселей,

каждая из которых будет проходить через нейронную сеть и на основе общей оценки всех частей будет выдаваться вердикт о качестве фотографии.

Распознавание же ночных сцен невозможно по отдельным частям фотографии, потому что такое распознавание требует контекста всей сцены. Например, на одной фотографии темным может быть звездное небо, а светлым - здание на переднем плане. Если пытаться распознать ночь «по частям», то части со звездным небом будут распознаваться как ночные, а части со зданием - как не ночные. В общем случае сложно сказать, какая часть изображения должна быть «ночной», чтобы считать всю фотографию ночной, в отличие от других меток *under* и *over*, по которым в сообществе фотографов существует примерное мнение о допустимом объеме недосвета или пересвета.

3.2 Расширение датасета

3.2.1 Выбор источника фотографий

Для обучения дополнительной головы *night* требовался набор изображений, где можно устойчиво отделять ночные сцены от обычных дневных/светлых. В рамках работы было принято решение собрать отдельный датасет из двух источников, чтобы обеспечить достаточное разнообразие сцен и получить явную разметку на уровне ночь/не ночь без ручной пометки каждого кадра.

В качестве источника ночных сцен использовался датасет ExDark. Его содержимое ориентировано на условия низкой освещённости, поэтому все изображения из этой папки рассматривались как примеры класса *night* = 1. Для этих файлов метки технических дефектов *blur* и *over* фиксировались в ноль (т.к. на этом датасете будет обучаться только голова *night*, головы *blur*, *over* и *under* обучаться не будут), а метка *under* назначалась случайно (0 или 1), поскольку цель данного источника - именно обучение распознаванию ночной сцены, а не точная оценка недосвета как дефекта.

В качестве источника «обычных» сцен использовался датасет MIT Indoor Scenes, преимущественно состоящий из светлых фотографий. Все изображения из этого источника считались примерами класса *night* = 0. Чтобы одновременно иметь в этом источнике как примеры без дефекта, так и примеры

с дефектом `under`, внутри каждой категории изображения делились на две равные части: для первой половины задавалась метка `under = 1`, для второй- `under = 0`. Метки `blur` и `over` также фиксировались в ноль. Такой подход позволяет получить контролируемую смесь «нормальных» и «искусственно недосвеченных» дневных сцен.

3.2.2 Генерация искаженных изображений

Искажения для изображений, помеченных как `under = 1` и `night = 0`, создавались автоматически в MATLAB. Для таких файлов применялся тип искажения 17 (`darken`) с уровнями 4–5, после чего изображение перезаписывалось без изменения имени. Логика генерации затемнения и вызов функции искажения были реализованы на основе ранее использованных MATLAB-скриптов генерации искажений для KADIS-700K: была переиспользована проверенная функция `imdist_generator` и общий принцип «тип искажения + уровень -> получение изменённого изображения». Одновременно в `night_labels.csv` обновлялись поля `dist_type` и `dist_level`, чтобы разметка соответствовала фактически применённому преобразованию.

В результате для хранения разметки был сформирован единый файл `night_labels.csv` с дополнительным полем `night`, которого нет в первом датасете.

После формирования `night_labels.csv` данные были разделены на обучающую и валидационную выборки. Скрипт, разделяющий датасет, полностью аналогичен скрипту для первой версии, за исключением доработки для работы с полем `night`.

В результате были получены файлы `night_train.csv` и `night_val.csv`, содержащие как изображения, так и разметку, при этом доля ночных сцен в обеих выборках сохраняется близкой к исходной. Это обеспечивает стабильную проверку качества головы `night`.

3.3 Третья итерация

Процесс обучения был разделён на три этапа. На первом этапе (5 эпох) общая часть была заморожена, и обучались только головы дефектов качества (`blur/under/over`); голова `night` в этот момент не участвовала в оптимизации. На втором этапе (ещё 5 эпох) выполнялась тонкая настройка верхних слоёв

бэббона: слои пакетной нормализации BatchNorm фиксировались, нижняя часть EfficientNet оставалась замороженной, а обучение шло с уменьшенной скоростью обучения. На третьем этапе (5 эпох) обучалась только голова night на отдельном датасете ночных/дневных сцен; при этом вклад остальных голов в функцию потерь отключался весами.

Качество голов blur/under/over оценивалось на обычной валидации val.csv. Итоговые значения не отличаются от значений второй итерации - что не удивительно, потому что изменения в обучение трех изначальных голов не вводились.

Качество головы night оценивалось отдельно на night_val.csv. Получены значения: TP=1414, FP=28, FN=59, TN=2782, что соответствует precision 0.9806, recall 0.9599, F1 0.9702 и accuracy 0.9797. Эти результаты показывают, что модель уверенно отделяет ночные сцены от дневных, а значит может использоваться как надёжный контекстный фильтр для under.

В процессе проверки оказалось, что среди всех ложных срабатываний under на val.csv в 66.86% случаев (234 из 350) ночная голова выдаёт night_pred = 1. В таком случае потенциально устраняется около 234 ложных срабатываний under, и остаётся порядка 116 FP, что уже сопоставимо с уровнем FP для over (106) и существенно ближе к «стабильным» головам (blur/over/night), FP которых равен около 30. Практически это подтверждает, что идея отдельной головы night является работоспособной именно как механизм снижения ложных браков по under.

Результаты проверки точности представлены на рисунке 9.


```

=====
[VAL.CSV] blur/under/over
=====
val samples: 8954
thresholds: blur=0.5 under=0.5 over=0.5

Micro-accuracy (3 метки): 0.9689
Exact-match accuracy (3 метки): 0.9119

[BLUR]
TP=2233 FP=31 FN=13 TN=6677
precision=0.9863 recall=0.9942 f1=0.9902 accuracy=0.9951

[UNDER]
TP=949 FP=350 FN=193 TN=7462
precision=0.7306 recall=0.8310 f1=0.7776 accuracy=0.9394

[OVER]
TP=947 FP=106 FN=142 TN=7759
precision=0.8993 recall=0.8696 f1=0.8842 accuracy=0.9723
2026-02-21 14:10:36.956711: W tensorflow/core/lib/png/png_
2026-02-21 14:10:57.686980: W tensorflow/core/lib/png/png_
2026-02-21 14:12:11.224365: W tensorflow/core/lib/png/png_
2026-02-21 14:12:11.224473: W tensorflow/core/lib/png/png_
2026-02-21 14:16:20.546156: I tensorflow/core/framework/lo

=====
[NIGHT_VAL.CSV] ONLY night head
=====
night_val samples: 4283
threshold: night=0.5

[NIGHT]
TP=1414 FP=28 FN=59 TN=2782
precision=0.9806 recall=0.9599 f1=0.9702 accuracy=0.9797

```

Рисунок 9 — Точность предсказаний третьей итерации

Однако график ключевых метрик ведет себя странно после 10-й эпохи. Это объясняется тем, что кривые строятся единым графиком для всех 15 эпох, однако начиная с 11-й эпохи меняется датасет и цель оптимизации: обучение переключается на задачу night, а метки blur/under/over на этом этапе не являются целевыми и фактически отключаются весами. Из-за этого значения метрик по головам качества на третьем этапе теряют смысл (они считаются на данных другого распределения, при иной схеме весов и вообще говоря датасет на night не размечен под over, under и blur), а общая AUC, построенная по всем меткам, также становится некорректной для сравнения с первыми 10 эпохами. Поэтому в третьей итерации интерпретацию обучения следует выполнять не по объединённому графику, а по итоговым метрикам проверки точности на соответствующих валидационных наборах: val.csv для blur/under/over и night_val.csv для night. Графики обучения представлены на рисунке 10.

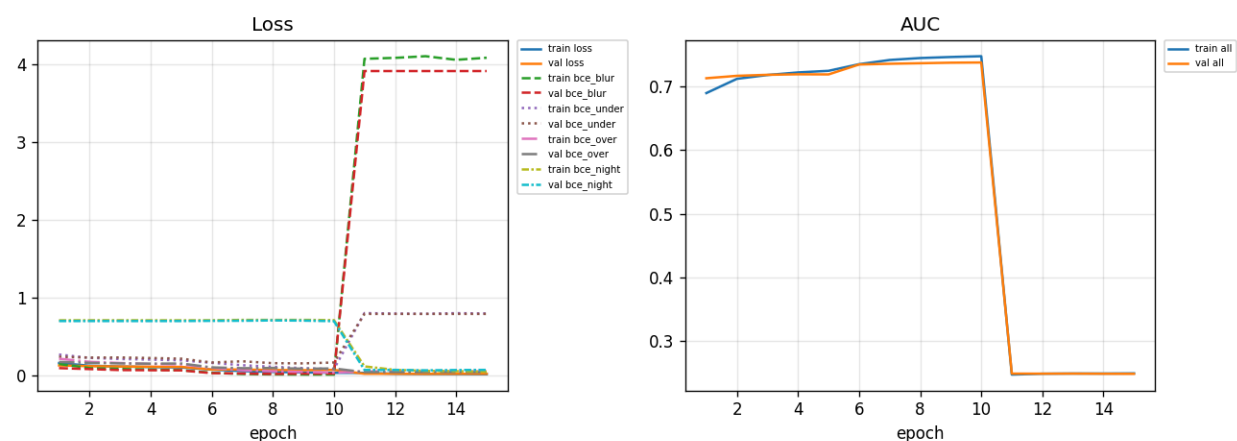


Рисунок 10 — Графики ключевых метрик третьей итерации

Таким образом, полученная версия модели является финальной для текущего этапа работ: она демонстрирует высокое качество по дефектам blur/over и приемлемое качество по under, а добавление головы night даёт практический механизм снижения ложных срабатываний under на ночных сценах в прикладной логике приложения - можно использовать правило: если кадр распознан как ночной, фильтрацию по under не применять. Текущая конфигурация подходит для встраивания в приложение ВКРБ.

ЗАКЛЮЧЕНИЕ

В ходе преддипломной практики была разработана нейросетевая подсистема для ВКРБ, предназначенная для автоматической оценки технического качества пользовательских фотографий. Базовая постановка задачи включала три технических дефекта: размытие (blur), недосвет (under) и пересвет (over). Для обучения была подготовлена выборка на основе KADIS-700K с отбором релевантных типов искажений, сформирован единый файл разметки labels.csv, выполнено разбиение train/val без утечек по сценам, а также реализован конвейер обучения в TensorFlow через tf.data.

В качестве основы модели использован EfficientNetB0 и архитектура с несколькими выходными головами. Во второй итерации была повышена стабильность обучения за счёт заморозки BatchNorm, увеличения количества нейронов у голов и частичного дообучения верхних слоёв бэкбона, расширен набор метрик для контроля качества по каждой метке. На валидационной выборке по дефектам качества получены значения: micro-accuracy = 0.9689 и exact-match accuracy = 0.9119; по головам blur достигнуты precision = 0.9863 и recall = 0.9942 (F1 = 0.9902), по over — precision = 0.8993 и recall = 0.8696 (F1 = 0.8842). Метка under оставалась наиболее сложной из-за ложных срабатываний (FP = 350 при пороге 0.5).

Для уменьшения ложных срабатываний under в финальной версии добавлена четвёртая голова night, распознающая ночные сцены. Голова night обучалась отдельно на специализированной разметке (night_train.csv/night_val.csv) без влияния на головы качества. По результатам проверки на night_val.csv достигнуты precision = 0.9806, recall = 0.9599, F1 = 0.9702 и accuracy = 0.9797, что подтверждает пригодность night для контекстной фильтрации. Дополнительно показано, что среди ложных срабатываний under на val.csv в 66.86% случаев модель одновременно предсказывает night_pred = 1 (234 из 350), то есть отключение/ослабление фильтрации under при ночной сцене позволяет существенно снизить число ложноположительных срабатываний и приблизить уровень FP по under к значениям других голов.

Таким образом, разработанная четырёхголовая модель обеспечивает высокое качество распознавания размытия и пересвета, а также предоставляет практический механизм уменьшения ошибок по недосвету за счёт учёта контекста сцены. Полученная версия может быть встроена в ВКРБ для автоматического отбора фотографий с возможностью настройки порогов и правил фильтрации на уровне прикладной логики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Мартынюк П.А. Лекции по Искусственному Интеллекту.
2. KADID-10k IQA Database [Электронный ресурс]. URL: <https://database.mmsp-kn.de/kadid-10k-database.html> (дата обращения: 12.02.2026).
3. Документация по MATLAB [Электронный ресурс]. URL: <https://www.mathworks.com/help/matlab/index.html> (дата обращения: 15.02.2026).
4. Использование моделей EfficientNet для классификации изображений [Электронный ресурс]. URL: <https://habr.com/ru/companies/sberbank/articles/828842/> (дата обращения: 17.02.2026).
5. Документация по TensorFlow [Электронный ресурс]. URL: <https://www.tensorflow.org/guide?hl=ru> (дата обращения: 18.02.2026).
6. Гордеев В.А., Булдакова Т.И. Исследование методов распознавания жестового языка при наличии негативных факторов // Математические методы в технологиях и технике. 2024. № 10. С. 77-81.
7. Gordeev V.A., Buldakova T.I. Research of Neural Network Models of Sign Language Recognition in the Presence of Negative Factors. In: Kravets A.G., Bolshakov A.A. (eds) Human-Centric Cyber-Society. Studies in Systems, Decision and Control, vol 639. 2026. Pp. 199-209. Springer, Cham. https://doi.org/10.1007/978-3-032-06787-6_16.